

Week 3 : Cognizant Hands on -- Spring REST using Spring Boot 3

Hands on 1

Create a Spring Web Project using Maven

Hands-on 1 – Create a Spring Web Project using Maven

Step 1: Open Spring Initializr

Go to:

<https://start.spring.io/>

Fill the details:

Field	Value
Project	Maven
Language	Java
Spring Boot	Latest Stable
Group	com.cognizant
Artifact	spring-learn
Name	spring-learn
Packaging	Jar
Java	17 (or as instructed)

Add Dependencies:

- Spring Boot DevTools
- Spring Web

Click Generate.

Step 2: Extract Project

Download

spring-learn.zip

Extract it into your Eclipse Workspace.

Example:

C:\Users\Username\eclipse-workspace\

Step 3: Build using Maven

Open Command Prompt.

Go inside the project.

Example

cd C:\Users\Username\eclipse-workspace\spring-learn

Run

mvn clean package

If Cognizant proxy is required, run

mvn clean package -Dhttp.proxyHost=proxy.cognizant.com -

Dhttp.proxyPort=6050 -

Dhttps.proxyHost=proxy.cognizant.com -

Dhttps.proxyPort=6050 -Dhttp.proxyUser=123456

Wait until

BUILD SUCCESS

Step 4: Import into Eclipse

Go to

File



Import



Maven



Existing Maven Projects



Browse



Select spring-learn folder



Finish

Step 5: Verify Folder Structure

You should see

spring-learn

src/main/java

src/main/resources

src/test/java

pom.xml

mvnw

SpringLearnApplication.java

Step 6: Add Log Statement

Open

SpringLearnApplication.java

Example

```
package com.cognizant.springlearn;
```

```
import org.springframework.boot.SpringApplication;
```

```
import
```

```
org.springframework.boot.autoconfigure.SpringBootApplication;
```

n;

@SpringBootApplication

public class SpringLearnApplication {

public static void main(String[] args) {

System.out.println("Spring Boot Application Started");

SpringApplication.run(SpringLearnApplication.class, args);

System.out.println("Application Running Successfully");

}

}

Step 7: Run Application

Right Click

SpringLearnApplication

↓

Run As

↓

Java Application

or

Run As



Spring Boot App

Console Output

Spring Boot Application Started

...

Tomcat started on port(s): 8080

Started SpringLearnApplication

Application Running Successfully

SME Walkthrough Answers

These are the questions your trainer may ask.

1. src/main/java

Contains all Java source code.

Example:

SpringLearnApplication.java

Controller classes

Service classes

Model classes

2. src/main/resources

Contains configuration files.

Example

application.properties

application.yml

static/

templates/

3. src/test/java

Contains JUnit test classes.

Used to test the application.

Example

SpringLearnApplicationTests.java

4. SpringLearnApplication.java

Main class of the Spring Boot project.

Contains

```
public static void main(String[] args)
```

which starts the Spring Boot application.

```
SpringApplication.run(SpringLearnApplication.class, args);
```

This loads the Spring container and starts the embedded Tomcat server.

5. Purpose of @SpringBootApplication

It is a combination of three annotations:

@SpringBootConfiguration

Marks it as a configuration class.

@EnableAutoConfiguration

Automatically configures Spring Boot.

@ComponentScan

Scans all components like

- Controller
 - Service
 - Repository
-

6. pom.xml Walkthrough

The pom.xml file manages the Maven project configuration.

Important sections include:

Project Information

`<groupId>com.cognizant</groupId>`

`<artifactId>spring-learn</artifactId>`

`<version>0.0.1-SNAPSHOT</version>`

Parent

spring-boot-starter-parent

Provides default Spring Boot dependency versions.

Dependencies

spring-boot-starter-web

For developing web applications.

spring-boot-devtools

Supports automatic restart during development.

spring-boot-starter-test

Provides JUnit and testing libraries.

Build Plugin

spring-boot-maven-plugin

Used to build and run the Spring Boot application.

Exercise 4

1. Project Structure

```
spring-learn
|
├── src/main/java
│   ├── com/cognizant/springlearn
│   │   ├── Country.java
│   │   └── SpringLearnApplication.java
└── src/main/resources
    └── country.xml
```

2. country.xml

Create country.xml inside src/main/resources

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-
beans.xsd">
```

```
  <bean id="country" class="com.cognizant.springlearn.Country">
    <property name="code" value="IN"/>
    <property name="name" value="India"/>
  </bean>
```

```
</beans>
```

3. Country.java

```
package com.cognizant.springlearn;
```

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```

```
public class Country {
```

```
    private static final Logger LOGGER =  
    LoggerFactory.getLogger(Country.class);
```

```
    private String code;  
    private String name;
```

```
    public Country() {  
        LOGGER.debug("Inside Country Constructor.");  
    }
```

```
    public String getCode() {  
        LOGGER.debug("Inside getCode()");  
        return code;  
    }
```

```
    public void setCode(String code) {  
        LOGGER.debug("Inside setCode()");  
        this.code = code;  
    }
```

```
    public String getName() {  
        LOGGER.debug("Inside getName()");  
        return name;  
    }
```

```

public void setName(String name) {
    LOGGER.debug("Inside setName()");
    this.name = name;
}

@Override
public String toString() {
    return "Country [code=" + code + ", name=" + name + "];"
}

}

```

4. SpringLearnApplication.java

```

package com.cognizant.springlearn;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import
org.springframework.context.support.ClassPathXmlApplicationContex
t;

@SpringBootApplication
public class SpringLearnApplication {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(SpringLearnApplication.class);

```

```
public static void main(String[] args) {  
  
    SpringApplication.run(SpringLearnApplication.class, args);  
  
    displayCountry();  
}  
  
public static void displayCountry() {  
  
    ApplicationContext context =  
        new ClassPathXmlApplicationContext("country.xml");  
  
    Country country = context.getBean("country", Country.class);  
  
    LOGGER.debug("Country : {}", country.toString());  
}  
}
```

5. application.properties

Create or open

src/main/resources/application.properties

Add

logging.level.root=INFO

logging.level.com.cognizant.springlearn=DEBUG

6. Expected Output

Inside Country Constructor.

Inside setCode()

Inside setName()

Country : Country [code=IN, name=India]

SME Questions & Answers

1. What is <bean> tag?

A bean represents an object managed by the Spring IoC Container.

Example

```
<bean id="country" class="com.cognizant.springlearn.Country"/>
```

2. What is id attribute?

It gives a unique name to the bean.

Example

```
id="country"
```

Retrieve using

```
context.getBean("country");
```

3. What is class attribute?

Specifies the fully qualified class name.

Example

```
class="com.cognizant.springlearn.Country"
```

Spring creates an object of this class.

4. What is <property> tag?

Used to inject values into bean properties.

Example

```
<property name="code" value="IN"/>
```

Spring internally calls

```
setCode("IN");
```

5. What is name attribute?

Specifies which setter method to call.

Example

```
<property name="name" value="India"/>
```

calls

```
setName("India");
```

6. What is value attribute?

It specifies the value assigned to the property.

Example

```
value="India"
```

7. What is ApplicationContext?

ApplicationContext is the Spring IoC Container. It loads configuration files, creates objects (beans), injects dependencies, and manages bean lifecycle.

Example

```
ApplicationContext context =  
new ClassPathXmlApplicationContext("country.xml");
```

8. What is ClassPathXmlApplicationContext?

It is an implementation of ApplicationContext that loads XML configuration files from the classpath.

Example

```
new ClassPathXmlApplicationContext("country.xml");
```

9. What happens when context.getBean() is invoked?

When

```
Country country = context.getBean("country", Country.class);
```

is executed:

1. Spring searches for the bean with ID "country".
2. If it does not already exist, it creates the object.
3. It calls the constructor.
4. It injects property values by calling setCode() and setName().
5. It stores the bean in the Spring container (singleton by default).
6. It returns the fully initialized object.

Hands-on 1: Hello World RESTful Web Service

Project Structure

spring-learn

src

```
└─ main
  │   └─ java
  │       └─ com.cognizant.springlearn
  │           └─ controller
  │               └─ HelloController.java
  │               └─ SpringLearnApplication.java
  │
  └─ resources
      └─ application.properties
  └─ test
```

pom.xml

HelloController.java

```
package com.cognizant.springlearn.controller;
```

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```

```
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
```

```
public class HelloController {
```

```
    private static final Logger LOGGER =
```

```
        LoggerFactory.getLogger(HelloController.class);
```

```
    @GetMapping("/hello")
```

```
    public String sayHello() {
```

```
        LOGGER.info("START");
```

```
String message = "Hello World!!";

LOGGER.info("END");

return message;
}

}

application.properties

server.port=8083

logging.level.com.cognizant.springlearn=DEBUG
```

Output

URL

http://localhost:8083/hello

Output

Hello World!!

Hands-on 2: REST - Country Web Service

Project Structure

spring-learn

src

```
└─main
  └─java
    └─com.cognizant.springlearn
```

```

|   |   |   | controller
|   |   |   |   | CountryController.java
|   |   |   |   | Country.java
|   |   |   |   | SpringLearnApplication.java
|   |   |   |
|   |   |   | resources
|   |   |   |   | application.properties
|   |   |   |   | country.xml
|   |   |   |
|   |   |   | test

```

country.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="
            http://www.springframework.org/schema/beans
            https://www.springframework.org/schema/beans/spring-
beans.xsd">
```

```
<bean id="country" class="com.cognizant.springlearn.Country">
  <property name="code" value="IN"/>
  <property name="name" value="India"/>
</bean>
```

</beans>

Country.java

```
package com.cognizant.springlearn;
```

```
public class Country {
```

```
private String code;
private String name;

public Country() {
}

public String getCode() {
    return code;
}

public void setCode(String code) {
    this.code = code;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}
}
```

CountryController.java

```
package com.cognizant.springlearn.controller;

import org.springframework.context.ApplicationContext;
import
org.springframework.context.support.ClassPathXmlApplicationConte
xt;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
```

```

import com.cognizant.springlearn.Country;

@RestController
public class CountryController {

    @RequestMapping("/country")
    public Country getCountryIndia() {

        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");

        return context.getBean("country", Country.class);
    }
}

```

Output

```

http://localhost:8083/country
{
  "code": "IN",
  "name": "India"
}

```

Hands-on 3: REST - Get Country Based on Country Code

Project Structure

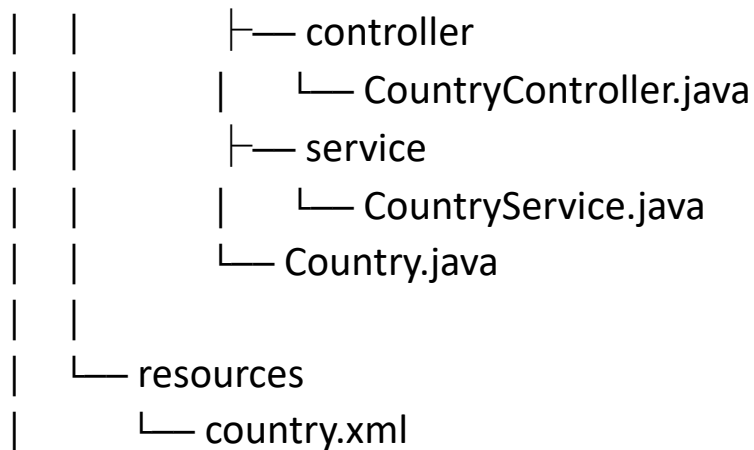
spring-learn

src

```

├── main
│   ├── java
│   │   └── com.cognizant.springlearn

```



country.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-
beans.xsd">
```

```
<bean id="countryList" class="java.util.ArrayList">
  <constructor-arg>
    <list>
```

```
<bean class="com.cognizant.springlearn.Country">
  <property name="code" value="IN"/>
  <property name="name" value="India"/>
</bean>
```

```
<bean class="com.cognizant.springlearn.Country">
  <property name="code" value="US"/>
  <property name="name" value="United States"/>
</bean>
```

```
<bean class="com.cognizant.springlearn.Country">
    <property name="code" value="JP"/>
    <property name="name" value="Japan"/>
</bean>
```

```
<bean class="com.cognizant.springlearn.Country">
    <property name="code" value="DE"/>
    <property name="name" value="Germany"/>
</bean>
```

```
</list>
</constructor-arg>
</bean>
```

```
</beans>
```

CountryService.java

```
package com.cognizant.springlearn.service;
```

```
import java.util.List;
```

```
import org.springframework.context.ApplicationContext;
```

```
import
```

```
org.springframework.context.support.ClassPathXmlApplicationConte
xt;
```

```
import org.springframework.stereotype.Service;
```

```
import com.cognizant.springlearn.Country;
```

```
@Service
```

```
public class CountryService {
```

```
    public Country getCountry(String code) {
```

```

ApplicationContext context =
    new ClassPathXmlApplicationContext("country.xml");

List<Country> list =
    (List<Country>) context.getBean("countryList");

for (Country country : list) {

    if (country.getCode().equalsIgnoreCase(code)) {
        return country;
    }
}

return null;
}

}

```

CountryController.java

```

package com.cognizant.springlearn.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

import com.cognizant.springlearn.Country;
import com.cognizant.springlearn.service.CountryService;

@RestController
public class CountryController {

```

```

@Autowired
private CountryService countryService;

@GetMapping("/countries/{code}")
public Country getCountry(@PathVariable String code) {

    return countryService.getCountry(code);
}
}

```

Output

URL

http://localhost:8083/countries/in

```

{
  "code": "IN",
  "name": "India"
}

```

Hands-on: Create Authentication Service that Returns JWT

Project Structure

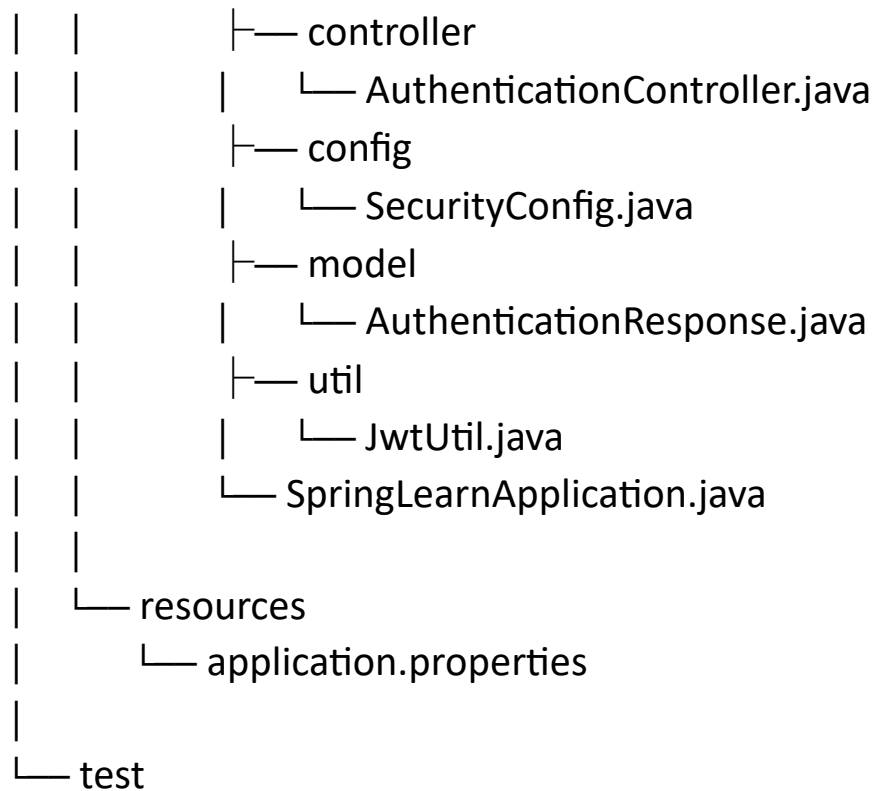
spring-learn

src

```

├── main
│   ├── java
│   │   └── com.cognizant.springlearn

```



pom.xml

1. AuthenticationResponse.java

```
package com.cognizant.springlearn.model;
```

```
public class AuthenticationResponse {
```

```
private String token;
```

```
public AuthenticationResponse() {
}
```

```
public AuthenticationResponse(String token) {
    this.token = token;
}
```

```
public String getToken() {  
    return token;  
}  
  
public void setToken(String token) {  
    this.token = token;  
}  
}
```

2. JwtUtil.java

```
package com.cognizant.springlearn.util;  
  
import java.util.Date;  
  
import io.jsonwebtoken.Jwts;  
import io.jsonwebtoken.SignatureAlgorithm;  
  
public class JwtUtil {  
  
    private static final String SECRET = "secretkey";  
  
    public static String generateToken(String username) {  
  
        return Jwts.builder()  
            .setSubject(username)  
            .setIssuedAt(new Date())  
            .signWith(SignatureAlgorithm.HS256, SECRET)  
            .compact();  
    }  
}
```

3. AuthenticationController.java

```
package com.cognizant.springlearn.controller;

import java.nio.charset.StandardCharsets;
import java.util.Base64;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RestController;

import com.cognizant.springlearn.model.AuthenticationResponse;
import com.cognizant.springlearn.util.JwtUtil;

@RestController
public class AuthenticationController {

    @GetMapping("/authenticate")
    public AuthenticationResponse authenticate(
        @RequestHeader("Authorization") String authHeader) {

        String encoded = authHeader.substring(6);

        byte[] decodedBytes = Base64.getDecoder().decode(encoded);

        String credentials = new String(decodedBytes,
StandardCharsets.UTF_8);

        String username = credentials.split(":")[0];

        String token = JwtUtil.generateToken(username);

        return new AuthenticationResponse(token);
    }
}
```

```
}  
  
}
```

4. SecurityConfig.java

```
package com.cognizant.springlearn.config;  
  
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.Configuration;  
import org.springframework.security.config.Customizer;  
import  
org.springframework.security.config.annotation.web.builders.HttpSec  
urity;  
import org.springframework.security.web.SecurityFilterChain;  
  
@Configuration  
public class SecurityConfig {  
  
    @Bean  
    public SecurityFilterChain securityFilterChain(HttpSecurity http)  
        throws Exception {  
  
        http  
            .csrf(csrf -> csrf.disable())  
            .authorizeHttpRequests(auth -> auth  
                .requestMatchers("/authenticate").permitAll()  
                .anyRequest().authenticated())  
            .httpBasic(Customizer.withDefaults());  
  
        return http.build();  
    }  
}
```

5. application.properties

server.port=8090

6. pom.xml Dependency

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt</artifactId>
  <version>0.9.1</version>
</dependency>
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

7. Run Command

```
curl -s -u user:pwd http://localhost:8090/authenticate
```

Output

```
{
  "token":"eyJhbGciOiJIUzI1NiJ9.eyJzdWliOiJ1c2Vy..."
}
```

SME Questions & Answers

1. Why Authentication Controller?

It accepts user credentials and returns a JWT token after successful authentication.

2. Why Authorization Header?

It carries the username and password encoded using Basic Authentication.

Example:

Authorization: Basic dXNlcjpwd2Q=

3. Why Base64 Decode?

The username and password are Base64 encoded in the Authorization header. They must be decoded before use.

4. Why JWT?

JWT (JSON Web Token) securely transfers user identity between client and server.

5. What does generateToken() do?

It creates a signed JWT containing:

- Subject (username)
- Issued time
- Digital signature

6. Why SecurityConfig?

It configures Spring Security, enables HTTP Basic Authentication, and allows access to the /authenticate endpoint.

This covers the mandatory implementation for the "Create Authentication Service that Returns JWT" hands-on.

Hands-on: Creating Microservices for Account and Loan

Account Microservice

Project Structure

account

src

```
|— main
|   |— java
|   |   └─ com.cognizant.account
|   |       └─ controller
|   |           └─ AccountController.java
|   |           └─ model
|   |               └─ Account.java
|   |               └─ AccountApplication.java
|   └─ resources
|       └─ application.properties
└─ test
```

pom.xml

Account.java

```
package com.cognizant.account.model;
```

```
public class Account {
```

```
private String number;  
private String type;  
private double balance;
```

```
public Account() {  
}
```

```
public Account(String number, String type, double balance) {  
    this.number = number;  
    this.type = type;  
    this.balance = balance;  
}
```

```
public String getNumber() {  
    return number;  
}
```

```
public void setNumber(String number) {  
    this.number = number;  
}
```

```
public String getType() {  
    return type;  
}
```

```
public void setType(String type) {  
    this.type = type;  
}
```

```
public double getBalance() {  
    return balance;  
}
```

```
    public void setBalance(double balance) {  
        this.balance = balance;  
    }  
  
}
```

AccountController.java

```
package com.cognizant.account.controller;  
  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.PathVariable;  
import org.springframework.web.bind.annotation.RestController;  
  
import com.cognizant.account.model.Account;  
  
@RestController  
public class AccountController {  
  
    @GetMapping("/accounts/{number}")  
    public Account getAccount(@PathVariable String number) {  
  
        return new Account(number, "Savings", 234343);  
  
    }  
  
}
```

application.properties

```
server.port=8080
```

URL

http://localhost:8080/accounts/00987987973432

Output

```
{  
  "number":"00987987973432",  
  "type":"Savings",  
  "balance":234343  
}
```

Loan Microservice

Project Structure

loan

src

```
├── main  
│   ├── java  
│   │   └── com.cognizant.loan  
│   │       ├── controller  
│   │       │   └── LoanController.java  
│   │       ├── model  
│   │       │   └── Loan.java  
│   │       └── LoanApplication.java  
│   └── resources  
│       └── application.properties  
└── test
```

pom.xml

Loan.java

```
package com.cognizant.loan.model;
```

```
public class Loan {
```

```
    private String number;
```

```
    private String type;
```

```
    private double loan;
```

```
    private double emi;
```

```
    private int tenure;
```

```
    public Loan() {
```

```
    }
```

```
    public Loan(String number, String type, double loan, double emi,  
int tenure) {
```

```
        this.number = number;
```

```
        this.type = type;
```

```
        this.loan = loan;
```

```
        this.emi = emi;
```

```
        this.tenure = tenure;
```

```
    }
```

```
    public String getNumber() {
```

```
        return number;
```

```
    }
```

```
    public void setNumber(String number) {
```

```
        this.number = number;
```

```
    }
```

```
public String getType() {  
    return type;  
}  
  
public void setType(String type) {  
    this.type = type;  
}  
  
public double getLoan() {  
    return loan;  
}  
  
public void setLoan(double loan) {  
    this.loan = loan;  
}  
  
public double getEmi() {  
    return emi;  
}  
  
public void setEmi(double emi) {  
    this.emi = emi;  
}  
  
public int getTenure() {  
    return tenure;  
}  
  
public void setTenure(int tenure) {  
    this.tenure = tenure;  
}  
}
```

LoanController.java

```
package com.cognizant.loan.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

import com.cognizant.loan.model.Loan;

@RestController
public class LoanController {

    @GetMapping("/loans/{number}")
    public Loan getLoan(@PathVariable String number) {

        return new Loan(number, "Car", 400000, 3258, 18);

    }

}
```

application.properties

```
server.port=8081
```

URL

<http://localhost:8081/loans/H00987987972342>

Output

```
{  
  "number":"H00987987972342",  
  "type":"Car",  
  "loan":400000,  
  "emi":3258,  
  "tenure":18  
}
```

Maven Build Command

mvn clean package

Testing URLs

Account Service

<http://localhost:8080/accounts/00987987973432>

Response

```
{  
  "number":"00987987973432",  
  "type":"Savings",  
  "balance":234343  
}
```

Loan Service

<http://localhost:8081/loans/H00987987972342>

Response

```
{  
  "number":"H00987987972342",  
  "type":"Car",
```

```
"loan":400000,  
"emi":3258,  
"tenure":18  
}
```

SME Viva Questions & Answers

1. What is a Microservice?

A microservice is a small, independent application that performs a single business function and can be developed, deployed, and scaled independently.

2. Why create separate Account and Loan services?

To separate different business functionalities. Each service can run independently and communicate with other services when required.

3. Why does the Loan service use port 8081?

The Account service already uses the default port **8080**. Running both on the same port causes a "**Port already in use**" error, so the Loan service is configured to use **8081**.

4. How do you change the server port?

By adding the following property to application.properties:

```
server.port=8081
```

5. Which annotation is used to create REST APIs?

```
@RestController
```

6. Which annotation maps a GET request?

```
@GetMapping
```

7. Which annotation retrieves values from the URL?

```
@PathVariable
```

